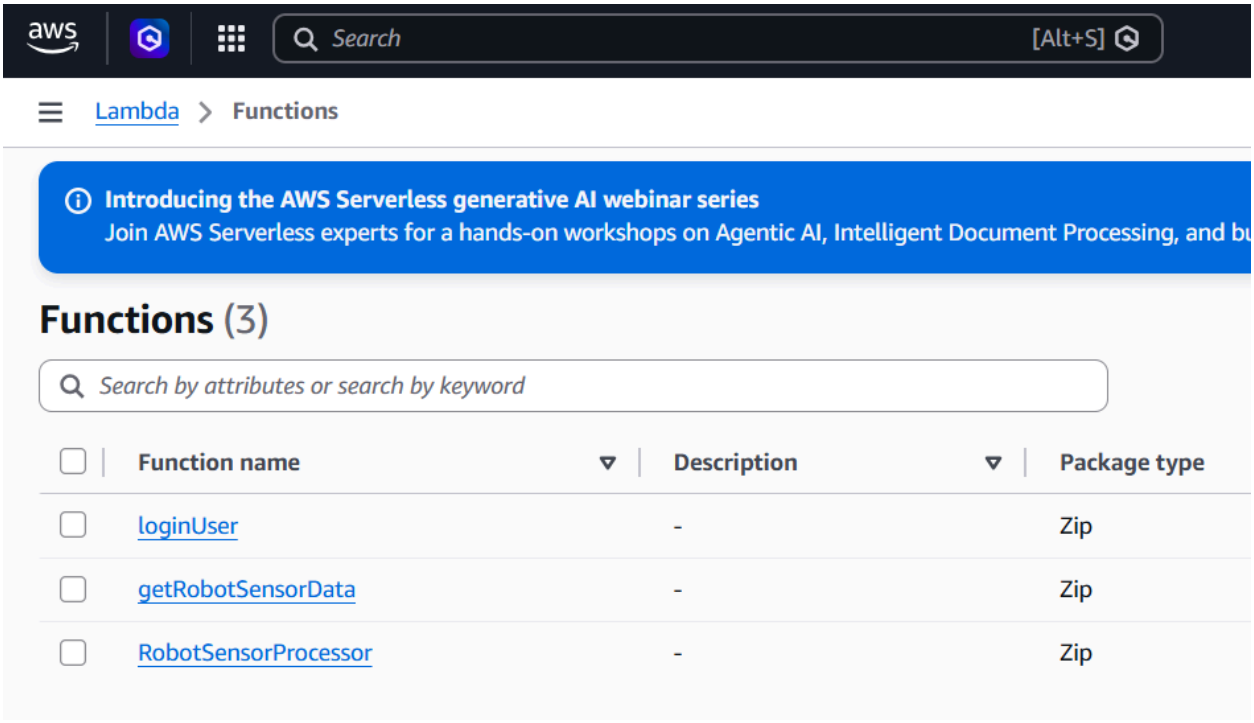
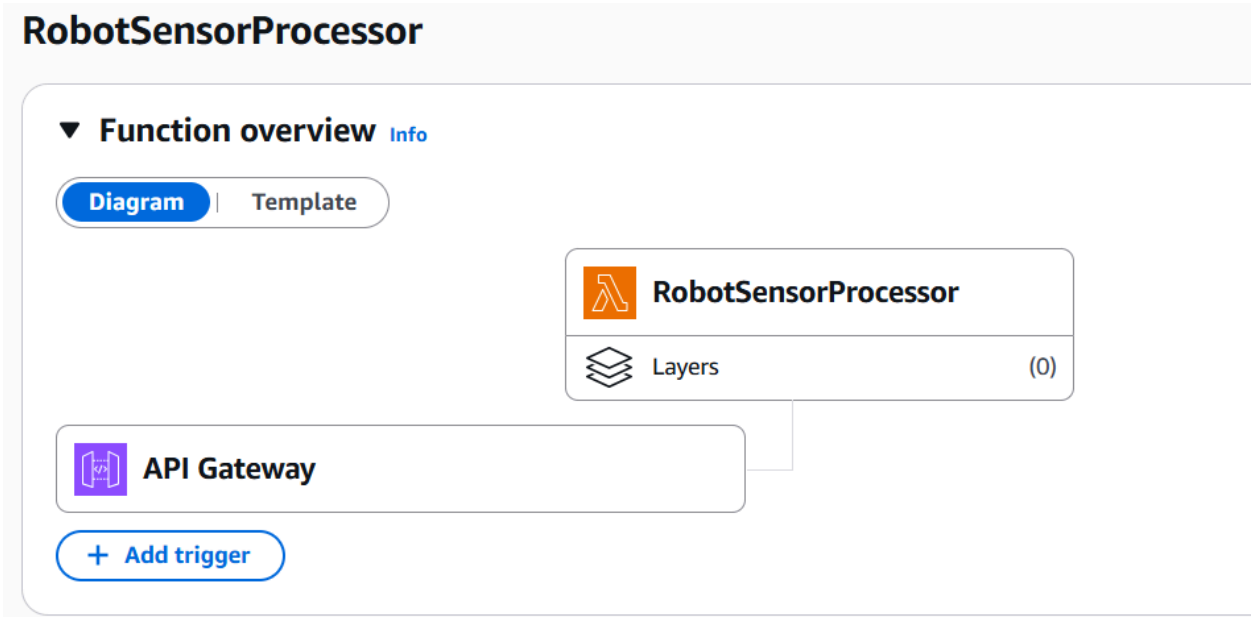
AWS Integration and Code





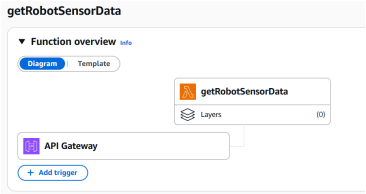
Post data code of aws lamda

```
import json
import boto3
import time

# Connect DynamoDB
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('robot_sensor_data_v2')

# Connect SNS
sns = boto3.client('sns')

TOPIC_ARN = "arn:aws:sns:ap-south-1:509194952926:Robot_fire_a
lerts"

def lambda_handler(event, context):

    try:

        # Handle API Gateway body
        if "body" in event:
            data = json.loads(event["body"])
        else:
            data = event

        device_id = data["device_id"]
        temperature = data["temperature"]
        gas = data["gas"]
        smoke = data["smoke"]
        flame = data["flame"]

        status = "NORMAL"

        # ALARM condition
        if flame == 1 or gas > 400 or smoke > 300 or temperat
ure > 60:
```

```
            status = "ALARM"

        # WARNING condition
        elif gas > 250 or smoke > 180 or temperature > 45:
            status = "WARNING"

        # Store data in DynamoDB
        table.put_item(
            Item={
                "device_id": device_id,
                "timestamp": int(time.time()),
                "temperature": temperature,
                "gas": gas,
                "smoke": smoke,
                "flame": flame,
                "status": status
            }
        )

        # Send SNS alerts
        if status == "ALARM":

            message = f"""
🔥 FIRE ALERT

Device: {device_id}

Temperature: {temperature}
Gas Level: {gas}
Smoke Level: {smoke}
Flame Detected: {flame}

Immediate action required!
"""

            sns.publish(
```

```
                TopicArn=TOPIC_ARN,
                Message=message,
                Subject="🔥 Robot Fire ALERT"
            )

        elif status == "WARNING":

            message = f"""
⚠ WARNING

Device: {device_id}

Temperature: {temperature}
Gas Level: {gas}
Smoke Level: {smoke}

Possible fire risk detected.
Please monitor the situation.
"""

            sns.publish(
                TopicArn=TOPIC_ARN,
                Message=message,
                Subject="⚠ Robot Fire Warning"
            )

        return {
            "statusCode": 200,
            "body": json.dumps({
                "status": status
            })
        }

    except Exception as e:

        return {
```

```
            "statusCode": 500,
            "body": json.dumps({
                "error": str(e)
            })
        }
```

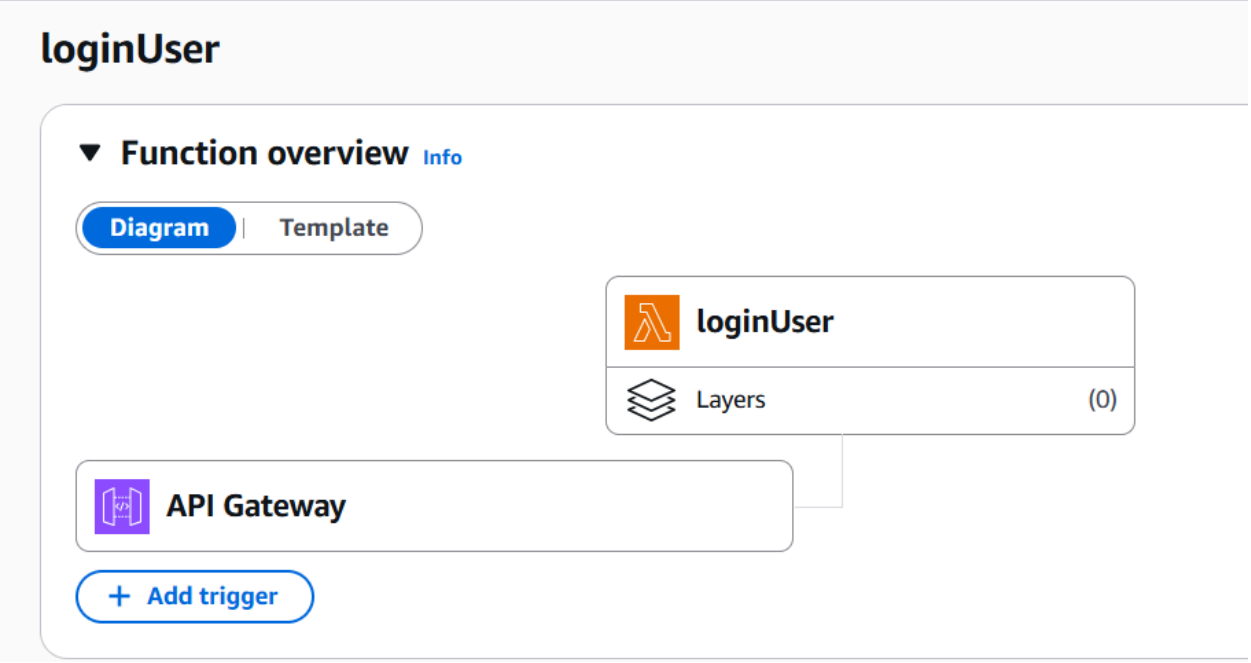


For get data from dynamo db table

```
import json
import boto3
from decimal import Decimal

# Connect DynamoDB
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('robot_sensor_data_v2')

# Convert Decimal → float
def decimal_default(obj):
    if isinstance(obj, Decimal):
        return float(obj)
    raise TypeError

def lambda_handler(event, context):

    try:
        # Get records
        response = table.scan()
        data = response['Items']

        return {
```

```
            "statusCode": 200,
            "headers": {
                "Access-Control-Allow-Origin": "*",
                "Access-Control-Allow-Headers": "*",
                "Access-Control-Allow-Methods": "*"
            },
            "body": json.dumps(data, default=decimal_default)
        }

    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({
                "error": str(e)
            })
        }
```

For login user authentication



```
import json
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Users')

def lambda_handler(event, context):

    try:

        body = json.loads(event['body'])

        username = body.get("username")
        password = body.get("password")

        if not username or not password:
            return {
                "statusCode":400,
                "headers":{
                    "Access-Control-Allow-Origin":"*",
                    "Access-Control-Allow-Headers":"*",
                    "Access-Control-Allow-Methods":"POST,OPTI
ONS"
                },
                "body":json.dumps({"message":"Username or pas
sword missing"})
            }

        response = table.get_item(Key={"username":username})

        if "Item" not in response:
            return {
                "statusCode":401,
                "headers":{
                    "Access-Control-Allow-Origin":"*",
```

```
                    "Access-Control-Allow-Headers":"*",
                    "Access-Control-Allow-Methods":"POST,OPTI
ONS"
                },
                "body":json.dumps({"message":"User not foun
d"})
            }

        user = response["Item"]

        if user["password"] == password:

            return {
                "statusCode":200,
                "headers":{
                    "Access-Control-Allow-Origin":"*",
                    "Access-Control-Allow-Headers":"*",
                    "Access-Control-Allow-Methods":"POST,OPTI
ONS"
                },
                "body":json.dumps({
                    "message":"Login successful",
                    "username":username
                })
            }

        else:

            return {
                "statusCode":401,
                "headers":{
                    "Access-Control-Allow-Origin":"*",
                    "Access-Control-Allow-Headers":"*",
                    "Access-Control-Allow-Methods":"POST,OPTI
ONS"
                },
```

```
                "body":json.dumps({"message":"Wrong passwor
d"})
            }

    except Exception as e:

        return {
            "statusCode":500,
            "headers":{
                "Access-Control-Allow-Origin":"*",
                "Access-Control-Allow-Headers":"*",
                "Access-Control-Allow-Methods":"POST,OPTIONS"
            },
            "body":json.dumps({"error":str(e)})
        }
```